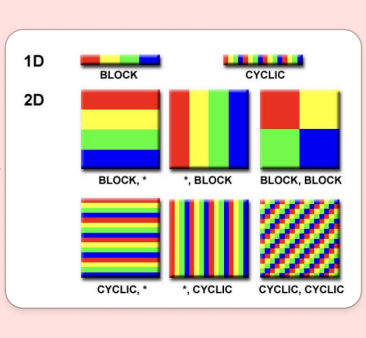
PCAM methodology

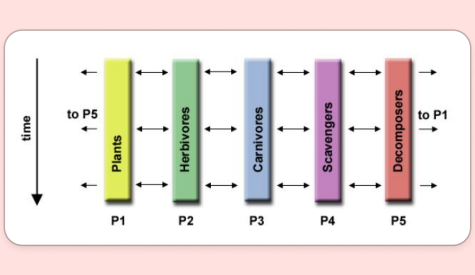
idea fondamentale: partire da un approccio astratto e indipendente dell'hardware, per poi scendere gradualmente nei dettagli specifici della macchina target

- **partitioning**
  - **goal:** expose opportunies for parallel execution within your problem
    - obiettivo: creare un numero di task molto superiore al numero di processori disponibili (decomposizione a "grana fine") per avere massima flessibilità in seguito
    - we are not worried about how many physical processors we have
    - we want to focus on defining a large number of small tasks
      - we call it **fine-grained decomposition**
      - **why do we want so many small tasks?**
        - it gives us flexibility
          - **ex.** if we create a partition with only 4 tasks because we have a 4-core CPU today, our program won't scale when we buy a 64-core machine tomorrow
        - a good partition divides both the computation and the data into small, manageable pieces
  - there are two main approaches to breaking down a problem
    - **domain decomposition**
      - dividiamo i dati in blocchi e associamo un task a ogni blocco
      - **focus:** on the data associated with the problem
        - we look at our dataset - whether it's a matrix, a grid or a list - and we chop it up
        - each parallel task it then responsible for working on just one specific portion of that data
      - ideally, we want these portions to be **small and of equal size** to ensure the workload is balanced later on
      - **example.** we can see different ways to slice this data

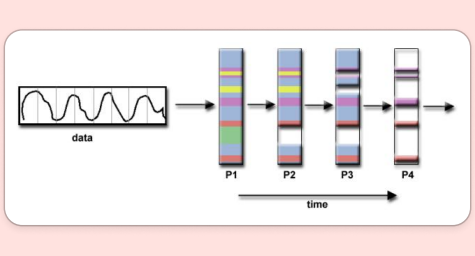


  - **1D decomposition**
    - slice an array into **blocks** (contiguous chunks)
    - assign elements in a **cyclic fashion**
  - **2D decomposition**
    - slice by rows or by columns
    - create 2D blocks (sub-matrices)
    - mix strategies, using a blockdistribution for rows and a cyclic distribution for columns

- **functional decomposition**
  - dividiamo il lavoro in base alle diverse funzioni da calcolare
  - **focus:** we focus on the **computation** (the work that need to be done)
    - we decompose the problem according to the specific functions or instructions that need to be executed
    - imagine the problem instruction set as a collection of different jobs; each task takes on a specific job
  - **example 1:** ecosystem modeling



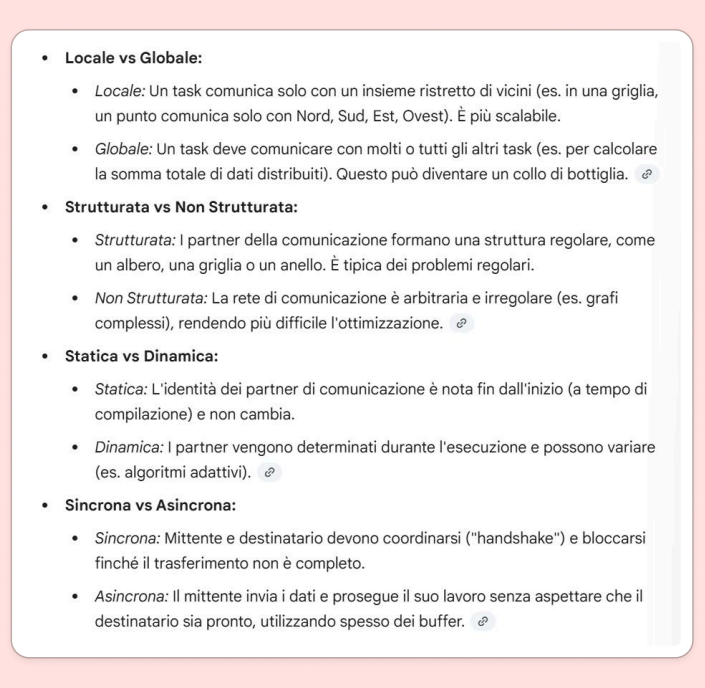
  - we have different groups: Plants, Herbivores, Carnivores, Scavengers and Decomposers
  - in a functional decomposition, each task calculates the population of just **one group**
    - however, these groups are linked - hervivores eat plants, carnivores eat herbivores
    - so, as time progresses, each task calculates its current state and then exhanges information with its neighbors before moving to the next time step
- **example 2:** signal processing



  - suppose an audio signal need to pass through four distinct filters
    - we can assign a separate task to each filter (P1, P2, P3, P4)
  - this creates a pipeline
    - when the first segment of data passes through the first filter, it moves to the second
    - while the second filter is processing that first segment, the first filter is already busy processing the second segment of data
    - by the time the fourth segment enters the pipeline, all four tasks are busy simultaneously
- mixing these two approaches is possibile

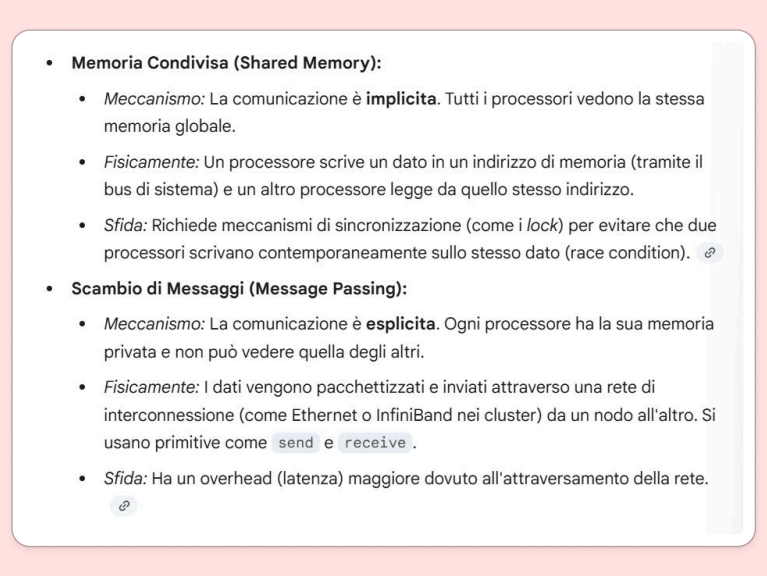
- **checklist** to ensure tour partition is robust
  - NB in questa fase abbiamo ignorato completamente quanti processori abbiamo realmente
  - **granularity** — does your partition define at leas an order of magnitude more tasks then there are processors?
    - if you have 100 processors, do you have 1000 tasks?
    - if not, you lose flexibility
  - **redundancy** — does it avoid redundant computation and storage?
    - if every task calculates the same thing, you aren't scaling efficiently
  - **balance** — are the tasks of comparable size?
    - if one task takes an hour and another takes a second, balacing the load later will be a nightmare
  - **scalability** — does the number of tasks grow as the problem size grows?
    - if your problem gets bigger, you parallelism sholf increase too

- **communication**
  - we have partitioned out problem into fine-grained tasks
    - we face a reality check: these tasks rarely exist in isolation
    - to solve the larger problem, they must interact, exchange data and coordinate
      - this brings us to the second phase of the PCAM methodolgy: **communication**
        - this phase is critical because as we move closer to the actual hardware implementation communication is often the bottleneck
          - if we aren't careful, the overhead of moving data around can completely nullify the benefits of running in parallel
  - first, we need to categorize the *types* of communication our algorithm requires
    - si analizza e si definisce come avviene il flusso di informazioni tra le task
    - **local vs global**
      - does a task only need to talk to its immediate neighbors (a "local" communication) or does it need to interact with all other tasks?
      - local communication is generally cheap and scalable — comunica solo con i suoi vicini
      - global communication involves many tasks simultaneously and can be a major performance hit — un task deve scambiare dati con tutti gli altri
    - **structured vs unstructured**
      - is the communication pattern regular, like a grid or a tree structure?
        - structured patterns are much easier to optimize
      - or is it arbitraty and unpredicable, forming a complex graph?
    - **static vs dynamic**
      - do we know who talks to whom at compile time (static)?
      - is the communication partner determined by runtime conditions (dynamic)?



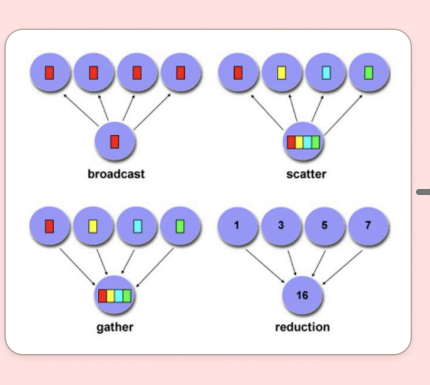
- **Locale vs Globale:**
  - *Locale:* Un task comunica solo con un insieme ristretto di vicini (es. in una griglia, un punto comunica solo con Nord, Sud, Est, Ovest). È più scalabile.
  - *Globale:* Un task deve comunicare con molti o tutti gli altri task (es. per calcolare la somma totale di dati distribuiti). Questo può diventare un collo di bottiglia.
- **Strutturata vs Non Strutturata:**
  - *Strutturata:* I partner della comunicazione formano una struttura regolare, come un albero, una griglia o un anello. È tipica dei problemi regolari.
  - *Non Strutturata:* La rete di comunicazione è arbitraria e irregolare (es. grafi complessi), rendendo più difficile l'ottimizzazione.
- **Statica vs Dinamica:**
  - *Statica:* L'identità dei partner di comunicazione è nota fin dall'inizio (a tempo di compilazione) e non cambia.
  - *Dinamica:* I partner vengono determinati durante l'esecuzione e possono variare (es. algoritmi adattivi).
- **Sincrona vs Asincrona:**
  - *Sincrona:* Mittente e destinatario devono coordinarsi ("handshake") e bloccarsi finché il trasferimento non è completo.
  - *Asincrona:* Il mittente invia i dati e prosegue il suo lavoro senza aspettare che il destinatario sia pronto, utilizzando spesso dei buffer.

  - we must look at how this communication physically happens, which depends heavily on the target architecture
    - about **visibility**
      - **explicit communication** — in models like Message Passing (MPI), the programmer must explicitly define the sender and the receiver
      - **implicit communication** — in Shared Memory systems, commynication happens simply by one task writing to a variable that another task reads
    - about **synchronization**
      - **synchronous** — the sender waits untile the receiver is ready and it blocks execution
      - **asynchronous**
        - tasks transfer data independently
        - the sender fires the data off and gets back to work immediately



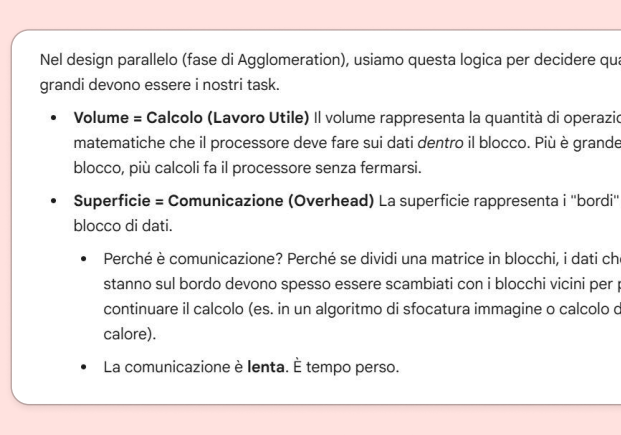
- **Memoria Condivisa (Shared Memory):**
  - *Meccanismo:* La comunicazione è **implicita**. Tutti i processori vedono la stessa memoria globale.
  - *Fisicamente:* Un processore scrive un dato in un indirizzo di memoria (tramite il bus di sistema) e un altro processore legge da quello stesso indirizzo.
  - *Sfida:* Richiede meccanismi di sincronizzazione (come i *lock*) per evitare che due processori scrivano contemporaneamente sullo stesso dato (race condition).
- **Scambio di Messaggi (Message Passing):**
  - *Meccanismo:* La comunicazione è **esplicita**. Ogni processore ha la sua memoria privata e non può vedere quella degli altri.
  - *Fisicamente:* I dati vengono pacchettizzati e inviati attraverso una rete di interconnessione (come Ethernet o InfiniBand nei cluster) da un nodo all'altro. Si usano primitive come `send` e `receive`.
  - *Sfida:* Ha un overhead (latenza) maggiore dovuto all'attraversamento della rete.

  - let's visualize the actual glow of data
    - **point-to-point**
      - this involves just two tasks - one sender and one receiver
      - while simple, this is often unfeasible for global operations because managing millions of individual handshakes is inefficient
    - **collective communication**
      - this involves a group of tasks working together
      - these are the standard building blocks of parallel programming



  - **broadcast:** one task has a piece of data (red square) and sends a copy to *everyone* else — un singolo dato invia lo *stesso dato* a tutti gli altri task
  - **scatter:** one tasks has a collection of different data (multicolored bar) and deals out a specific piece to each task — un task possiede un array di dati e ne invia un *pezzo diverso* a ciascun altro task
  - **gather:** everyone sends their distinct piece of data to one task, wich reassembles them — opposite of scatter
    - tutti i task inviano i loro pezzi di dati a un singolo task, che li raccoglie e li ricompone in un unico array ordinato
  - **reduction:** everyone has a value (numbers 1, 3, 5, 7) and they combine them (e.g., sum them up) to produce a single final result (16)
    - tutti i task inviano un valore ma questi non vengono semplicemente raccolti ma combinati → producono un singolo risultato finale nel task di destinazione
  - **managing overhead**
    - obiettivo è organizzare la comunicazione per minimizzare i colli di bottiglia, ma rimanendo ancora sul piano astratto
    - overhead comes from two sources
      - the time it takes to physically transmit the data
      - the time spent on synchronization (waiting for data to arrive)
    - **goal:** to minimize the number of channels and to overlap communication and computation
      - if a processor can do useful math *while* it is waiting for data to arrive from memory or the network, you are hiding the latency and increasing efficiency
  - **checklist** to ensure your communication is robust
    - **balance** — is the communication distributed equally?
      - if one task is doing all the talking (a bottleneck), your scalability is limited
    - **locality** — are most comunications local?
      - point-to-point communications with distant processors are expensive
    - **concurrency** — can communication operations happen in parallel?
    - **overlap** — can the computation proceed concurrently with communication?
      - if not, your processors will be idling

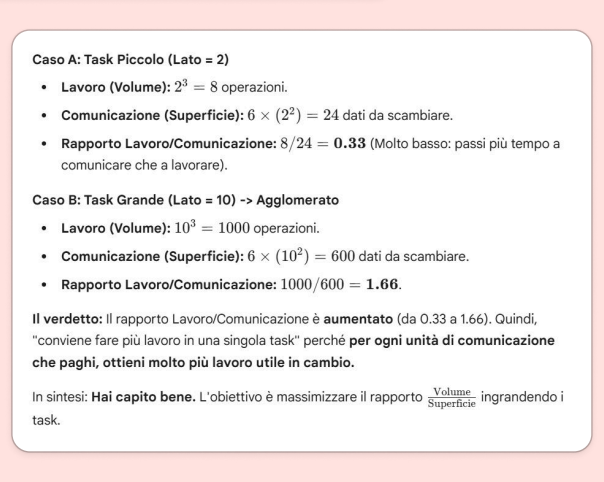
- **agglomeration**
  - we have decomposed our problem into many small tasks and defined how they talk to each other
    - ideally, in an abstract worlds with infinite processors, wewould be done
    - but in the real world, creating and managing a massive number of tiny tasks has a cost
      - the overhead of switching between thousand of threads or sending millions of tiny messages can destroy perdformance
      - spesso avere milioni di piccoli task crea troppo overhead di gestione e troppa comunicazione
  - **goal:** revisit our initial partition and optimize it for efficiency on a specific target architecture
    - this is the phase where we move from abstract parallel concepts to a concrete **real implementation**
  - the primary action in agglomeration is to reduce the number of tasks by combining them into larger ones
    - we move from a "fine-grained" decomposition to a "coarse-grained" one
    - **why do we do this?**
      - by grouping small tasks together
        - we eliminate the need to manage them individually
        - we often eliminate the communication that used to happen between them
      - if Task A and Task B are now part of the same larger task running on the same processor, they don't need to send network messages to each other anymore
        - they just share local memory
    - there are several ways to do this
      - l'obiettivo è migliorare le performance riducendo i costi di comunicazione → se due task comunicano molto tra di loro, conviene unirli in un unico task che gira sullo stesso processore, eliminando la necessità di trasmettere dati
      - **combining adjacent tasks:** instead of one task per grid point, you might assign a whole block of points to a single task
      - **dimensional reduction:** you might take a 3D grid of tasks and agglomerate them into 2D slices or 1D columns
      - **tree coalescing:** in a tree structure, you might group a parent node with its children into a single processing unit



Nel design parallelo (fase di Agglomeration), usiamo questa logica per decidere quanto grandi devono essere i nostri task.

- **Volume = Calcolo (Lavoro Utile)** Il volume rappresenta la quantità di operazioni matematiche che il processore deve fare sui dati *dentro* il blocco. Più è grande il blocco, più calcoli fa il processore senza fermarsi.
- **Superficie = Comunicazione (Overhead)** La superficie rappresenta i "bordi" del tuo blocco di dati.
  - Perché è comunicazione? Perché se dividi una matrice in blocchi, i dati che stanno sul bordo devono spesso essere scambiati con i blocchi vicini per poter continuare il calcolo (es. in un algoritmo di sfocatura immagine o calcolo del calore).
  - La comunicazione è **lenta**. È tempo perso.

  - the theoretical justification for agglomerations is often related to the **Surface-to-Volume ratio**
    - si cerca di aumentare la granularità per ottimizzare il rapporto tra calcolo e comunicazione
    - think of a task as a geometric shape
      - the "volume" represents the amount of **computation** (the work it does)
      - the "surface area" represents the **communication** (the boundary where it exchanges data with neighbors)
      - as you increase the size of the task (agglomerate), the volume (computation) grows much faster than the surface area (communication)
        - n^3 (volume) > n^2 (surface)



Caso A: Task Piccolo (Lato = 2)

- **Lavoro (Volume):** $2^3 = 8$ operazioni.
- **Comunicazione (Superficie):** $6 \times (2^2) = 24$ dati da scambiare.
- **Rapporto Lavoro/Comunicazione:** $8/24 = \mathbf{0.33}$ (Molto basso: passi più tempo a comunicare che a lavorare).

Caso B: Task Grande (Lato = 10) -> Agglomerato

- **Lavoro (Volume):** $10^3 = 1000$ operazioni.
- **Comunicazione (Superficie):** $6 \times (10^2) = 600$ dati da scambiare.
- **Rapporto Lavoro/Comunicazione:** $1000/600 = \mathbf{1.66}$.

Il verdetto: Il rapporto Lavoro/Comunicazione è **aumentato** (da 0.33 a 1.66). Quindi, "conviene fare più lavoro in una singola task" perché **per ogni unità di comunicazione che paghi, ottieni molto più lavoro utile in cambio.**

In sintesi: **Hai capito bene.** L'obiettivo è massimizzare il rapporto $\frac{\text{Volume}}{\text{Superficie}}$ ingrandendo i task.

    - this leads to a crucial shift in performance ratios
      - the **communication/computation ratio decreases:** you are doing more work for every byte of data you transmit
      - the **efficiency increases:** the processor spends more time calculating and less time waiting for data
      - the **parallelism decreases:** by merging tasks, tou have fewer total tasks available to distribute
        - this is the trade-off
  - **flexibility and replication**
    - agglomerations isn't just about merging but sometimes it involves **replication**
      - occasionally, it is actually faster to duplicate a calculation on multiple processors than to calculate it once and send the result across the network
      - if the cost of communication is high and the calculation is cheap, replicating the work improves performance
    - however, we must maintain **flexibility**
      - we shouldn't agglomerate down to exactly the number of physical processor (e.g., 4 tasks for 4 cores)
      - we usually want *more* tasks than processors to allow for
        - **load balancing:** if one processor finishes early, it can grab another task
        - **latency hiding:** while one task waits for memory, the processor can switch to another task
  - **checklist** to validate our communication
    - **communication** — has the increased locality actually reduced communication costs?
    - **scalability** — if we replicated data, does it compromise our ability to scale to larger problems?
    - **balance**
      - are the new, larger tasks still balanced?
      - if we created one giant tasks and three small ones, we've created a bottleneck
    - **concurrency**
      - is there still enough concurrency?
      - if we agglomerated too much, we might leave processors idle

- **mapping**
  - in this phase, we must specify exactly ehere each task will execute
    - fase finale nella quale assegniamo i task agglomerati alle risorse fisiche (processori, core, nodi del cluster)
    - we are assigning our agglomerated tasks to specific physical processors or threads
    - if we are working on a standard shared-memory system (like the multicore CPU in your laptop=, the operating system often handles a lot of this for you so it is less of a concern
      - however, on distributed-memory systems (like supercomputers or clusters), mapping is absolutely critical
  - **goal:** minimize the total execution in time
    - achieving this involves navigating a fundamental conflict between two competing strategies
      - **maximing concurrency:** to get the job done faster, we want to place tasks that can run simultaneously on different processors
        - enhances physical concurrency
        - ensures we are actually computing in parallel
      - **maximizing locality:** if two tasks talk to each other frequently, we want to place them on the **same processor**
        - this increases locality and eliminates the cost of sending data over the network
    - these two strategies conflict with each other
      - we have to balance computation speed with communication costs, while also respecting any resource limitations
      - dobbiamo bilanciare due obiettivi opposti:
        - massimizzare l'utilizzo → tenere tutti processori occupati
        - minimizzare la comunicazione → tenere vicini i task che si scambiano i dati
  - finding the perfect mapping is an **NP-complete problem**
    - we cannot solve it perfectly in a reasonable amount of time, so we rely on **heuristics** and load-balancing algorithms
    - we have to approaches
      - **static mapping**
        - we decide where every task goes *before* the program runs
        - great for regular predictable problems (like grid calculations)
        - too restrictive if the workload varies
      - **dynamic mapping (task scheduling)**
        - we assign tasks to processors at runtime
        - used when we have a functional decomposition with weak locallity requirements
          - tasks don't share much data
        - **goal:** keep the processors busy
          - we can implement this using centralized schemes (like a Manager/ Worker model= or decentralized ones
  - **checklist** strategies for mapping
    - **responsiveness**
      - is static mapping too rigid?
      - if the workload changes, will your program slow down?
    - **overhead** — is dynamic mapping too expensive?
      - if the scheduler takes more time to assign a task than the task takes to execute, you have a problem
    - **bottlenecks** — does your centralized scheduler create a bottleneck?
      - if 1,000 workers are asking one manager for work, the manager will crash
    - **coordination** — do dynamic load-balancing schemes require too much coordination?
      - moving tasks between processors to re-balance the load is expensive
    - **concurrency** — finally, are there actually enough tasks to keep all processors busy?
      - if not, you might have over-agglomerated in the previous step and processors will sit idle